

# assignment520

## DSA Assignment 520: 20260520模拟考

Updated 2026-05-20 16:47 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

### 说明:

#### 1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

## 1. 题目

### E04080: Huffman编码树

<http://cs101.openjudge.cn/practice/04080/>

思路:

用最小堆实现 Huffman编码树，叶子节点的权重路径和等于非叶子节点的权重和。

代码:

```
import heapq

def main():
    n = int(input())
    w = list(map(int, input().split()))
    heapq.heapify(w)
    total_wpl = 0
```

```

while len(w) > 1:
    left = heapq.heappop(w)
    right = heapq.heappop(w)
    combined = left + right
    total_wpl += combined
    heapq.heappush(w, combined)
print(total_wpl)

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#52355495提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```

import heapq

def main():
    n = int(input())
    w = list(map(int, input().split()))
    heapq.heapify(w)
    total_wpl = 0
    while len(w) > 1:
        left = heapq.heappop(w)
        right = heapq.heappop(w)
        combined = left + right
        total_wpl += combined
        heapq.heappush(w, combined)
    print(total_wpl)

if __name__ == '__main__':
    main()

```

基本信息

#: 52355495  
 题目: 04080  
 提交人: 25n2500011619(xq)  
 内存: 3640kB  
 时间: 30ms  
 语言: Python3  
 提交时间: 2026-04-21 15:49:24

## M05443: 兔子与樱花

dijkstra, Floyd-Warshall, <http://cs101.openjudge.cn/practice/05443/>

思路:

使用Floyd求出全图最短路。记录路径的方法是对每一组起点  $u$  和终点  $v$ ，记录起点下一步的点  $next\_node$ ，并在松弛操作时更新。

代码:

```

def main():
    inf = float('inf')

    p = int(input())
    id_node = [input().strip() for i in range(p)]
    node_id = {node : i for i, node in enumerate(id_node)}

```

```

dist = [[inf] * p for _ in range(p)]
next_node = [[-1] * p for _ in range(p)]
q = int(input())
for _ in range(q):
    edge = input().split()
    u, v = node_id[edge[0]], node_id[edge[1]]
    w = int(edge[2])
    dist[u][v] = w
    dist[v][u] = w
    next_node[u][v] = v
    next_node[v][u] = u

# Floyd - Warshall
for k in range(p):
    for u in range(p):
        if dist[u][k] == inf:
            continue
        for v in range(p):
            if dist[u][v] > dist[u][k] + dist[k][v]:
                dist[u][v] = dist[u][k] + dist[k][v]
                next_node[u][v] = next_node[u][k]

r = int(input())
for _ in range(r):
    line = input().split()
    src, dst = node_id[line[0]], node_id[line[1]]
    ans = []
    while src != dst:
        nxt = next_node[src][dst]
        ans.append(id_node[src] + f'->({dist[src][nxt]})->')
        src = nxt
    ans.append(id_node[dst])
    print(''.join(ans))

if __name__ == '__main__':
    main()

```

## 代码运行截图 (至少包含有"Accepted")

#52584130提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
def main():
    inf = float('inf')

    p = int(input())
    id_node = [input().strip() for i in range(p)]
    node_id = {node : i for i, node in enumerate(id_node)}

    dist = [[inf] * p for _ in range(p)]
    next_node = [[-1] * p for _ in range(p)]
    q = int(input())
    for _ in range(q):
        edge = input().split()
        u, v = node_id[edge[0]], node_id[edge[1]]
        w = int(edge[2])
        dist[u][v] = w
        dist[v][u] = w
        next_node[u][v] = v
        next_node[v][u] = u
```

基本信息

#: 52584130  
题目: 05443  
提交人: 25n2500011619(xq)  
内存: 3664kB  
时间: 23ms  
语言: Python3  
提交时间: 2026-05-19 18:23:04

## M20741: 两座孤岛最短距离

bfs, <http://cs101.openjudge.cn/practice/20741/>

思路:

先扩展一座孤岛，然后将上面的点加入起点；再做一次多起点的BFS，找两个岛之间的最短路径

代码:

```
from collections import deque

dx = [-1, 0, 1, 0]
dy = [0, 1, 0, -1]

def bfs1(n, sx, sy, matrix):
    q = [(sx, sy)]
    dist = [[-1] * n for _ in range(n)]
    dist[sx][sy] = 0
    s = []
    while q:
        zai
        for t in range(4):
            nx = x + dx[t]
            ny = y + dy[t]
            if 0 <= nx < n and 0 <= ny < n:
                if matrix[nx][ny] == '1' and dist[nx][ny] == -1:
                    dist[nx][ny] = 0
                    q.append((nx, ny))
```

```

        elif matrix[nx][ny] == '0':
            flag = True
        if flag: s.append((x, y))
    return dist, s

def bfs2(n, s, matrix, dist):
    q = deque(s)
    while q:
        x, y = q.popleft()
        for t in range(4):
            nx = x + dx[t]
            ny = y + dy[t]
            if 0 <= nx < n and 0 <= ny < n:
                if matrix[nx][ny] == '0':
                    if dist[nx][ny] == -1:
                        dist[nx][ny] = dist[x][y] + 1
                        q.append((nx, ny))
                elif dist[nx][ny] == -1:
                    return dist[x][y]
    return -1

def main():
    n = int(input())
    matrix = [input().strip() for _ in range(n)]
    sx, sy = 0, 0
    for i in range(n):
        for j in range(n):
            if matrix[i][j] == '1':
                sx, sy = i, j
                break

    dist, s = bfs1(n, sx, sy, matrix)

    ans = bfs2(n, s, matrix, dist)

    print(ans)

if __name__ == '__main__':
    main()

```

## 代码运行截图 (至少包含有"Accepted")

#52631441提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
from collections import deque

dx = [-1, 0, 1, 0]
dy = [0, 1, 0, -1]

def bfs1(n, sx, sy, matrix):
    q = [(sx, sy)]
    dist = [[-1] * n for _ in range(n)]
    dist[sx][sy] = 0
    s = []
    while q:
        x, y = q.pop()
        flag = False
        for t in range(4):
            nx = x + dx[t]
            ny = y + dy[t]
            if 0 <= nx < n and 0 <= ny < n:
                if matrix[nx][ny] == '1' and dist[nx][ny] == -1:
```

基本信息

#: 52631441  
题目: 20741  
提交人: 25n2500011619(xq)  
内存: 3760kB  
时间: 27ms  
语言: Python3  
提交时间: 2026-05-25 20:37:31

## M24637: 宝藏二叉树

dp, dfs <http://cs101.openjudge.cn/practice/24637/>

思路:

树形 dp，两个 dp 数组分别记录以 i 为根，选/不选 i 时整棵子树的最大值。从下往上更新。类似 O(n) 建堆方法，可以从 n // 2 处开始更新。

代码:

```
def main():
    n = int(input())
    tree = [0] + list(map(int, input().split()))
    dp = [[0] * 2 for _ in range(n + 5)]
    mid = n // 2
    for i in range(mid + 1, n + 1):
        dp[i][1] = tree[i]
    for i in range(mid, 0, -1):
        l = i << 1
        r = i << 1 | 1
        dp[i][1] = dp[l][0] + dp[r][0] + tree[i]
        dp[i][0] = max(dp[l]) + max(dp[r])
    print(max(dp[1]))

if __name__ == '__main__':
    main()
```

## 代码运行截图（至少包含有"Accepted"）

#52632073提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
def main():
    n = int(input())
    tree = [0] + list(map(int, input().split()))
    dp = [[0] * 2 for _ in range(n + 5)]
    mid = n // 2
    for i in range(mid + 1, n + 1):
        dp[i][1] = tree[i]
    for i in range(mid, 0, -1):
        l = i << 1
        r = i << 1 | 1
        dp[i][1] = dp[l][0] + dp[r][0] + tree[i]
        dp[i][0] = max(dp[l]) + max(dp[r])
    print(max(dp[1]))

if __name__ == '__main__':
    main()
```

基本信息

#: 52632073  
题目: 24637  
提交人: 25n2500011619(xq)  
内存: 3652kB  
时间: 23ms  
语言: Python3  
提交时间: 2026-05-25 21:44:08

## T02337: Catenyms

Eulerian Path, <http://cs101.openjudge.cn/practice/02337/>

思路:

对每个单词建立从第一个字母指向最后一个字母的有向边，题目转化为求欧拉路径。通过入度与出度判断是否存在合法的路径以及起点，然后使用Hierholzer算法，dfs 搜索，每次经过一条边即标记（等效删除），并在当前点无出边时将该点倒序加入答案路径。

为了满足题目对于最小字典序的要求，可以在输入的时候将单词按字典序排序。

为了避免重复走边，用 edge\_idx 数组记录每个节点走到哪条边。

代码

```
import sys

sys.setrecursionlimit(2000)

def main():

    t = int(input())
    output = []

    for _ in range(t):
        n = int(input())
        words = [input().strip() for _ in range(n)]
        words.sort()
        edges = [[] for _ in range(26)]
        in_deg = [0] * 26
```

```

out_deg = [0] * 26

for word in words:
    u = ord(word[0]) - 97
    v = ord(word[-1]) - 97
    edges[u].append((v, word))
    in_deg[v] += 1
    out_deg[u] += 1

start, end = -1, -1
is_invalid = False
for i in range(26):
    delta = in_deg[i] - out_deg[i]
    if delta == -1: # 入度大于出度
        if start != -1:
            is_invalid = True
            start = i
    elif delta == 1: # 入度小于出度
        if end != -1:
            is_invalid = True
            end = i
    elif delta:
        is_invalid = True

# print(chr(start + 97), chr(end + 97), is_invalid)
if is_invalid:
    output.append("***")
    continue

if start == -1:
    for i in range(26):
        if out_deg[i] > 0:
            start = i
            break

if start == -1: # 防御性编程：如果没有边
    output.append("***")
    continue

# Hierholzer 算法求欧拉路径
path = []
edge_idx = [0] * 26 # 记录每个节点走到哪条边了，避免重复走边

def dfs(u):
    while edge_idx[u] < len(edges[u]):
        v, word = edges[u][edge_idx[u]]

```



```

        edge_idx[u] += 1
        dfs(v)
        path.append(word)

    dfs(start)
    # 连通性校验：检查是否所有的单词都被用上了
    # 如果图是不连通的（存在分离的环），DFS 出来的路径长度会小于 n
    if len(path) < n:
        output.append("***")
    else:
        output.append(".".join(path[::-1]))

sys.stdout.write("\n".join(output))

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

#52636544提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

import sys

sys.setrecursionlimit(2000)
def main():

    t = int(input())
    output = []

    for _ in range(t):
        n = int(input())
        words = [input().strip() for _ in range(n)]
        words.sort()
        edges = [[] for _ in range(26)]
        in_deg = [0] * 26
        out_deg = [0] * 26

```

基本信息

#: 52636544  
 题目: 02337  
 提交人: 25n2500011619(xq)  
 内存: 4956kB  
 时间: 26ms  
 语言: Python3  
 提交时间: 2026-05-26 16:39:08

## T30878:力场叠加模拟

segment tree, lazy propagation, <http://cs101.openjudge.cn/practice/30878/>

思路:

含lazy tag的线段树，实现区间查询最大值

代码

```

import sys

# 设置递归深度和快速输入

```

```

input = sys.stdin.readline
inf = float('inf')

class SegmentTree:
    def __init__(self, n, nums):
        self.n = n
        self.nums = nums
        self.tree = [0] * (4 * self.n)
        self.tag = [0] * (4 * self.n)
        # self.build(1, 1, n)

    def push_up(self, node):
        return

    def push_down(self, node):
        if self.tag[node] and (node << 1 | 1) < (self.n << 2):
            self.tree[node << 1] += self.tag[node]
            self.tag[node << 1] += self.tag[node]
            self.tree[node << 1 | 1] += self.tag[node]
            self.tag[node << 1 | 1] += self.tag[node]
            self.tag[node] = 0
        return

    def build(self, node, l, r):
        if l == r:
            self.tree[node] = self.nums[l]
            return
        mid = (l + r) >> 1
        self.build(node << 1, l, mid)
        self.build(node << 1 | 1, mid + 1, r)
        self.tree[node] = max(self.tree[node << 1], self.tree[node << 1 | 1])

    def update_range(self, node, start, end, l, r, value):
        if start <= l and r <= end:
            self.tree[node] += value
            self.tag[node] += value
            return
        if end < l or start > r:
            return
        mid = (l + r) >> 1
        self.push_down(node)
        self.update_range(node << 1, start, end, l, mid, value)
        self.update_range(node << 1 | 1, start, end, mid + 1, r, value)
        self.tree[node] = max(self.tree[node << 1], self.tree[node << 1 | 1])
        return

```

```

def query(self, node, start, end, l, r):
    if start <= l and r <= end:
        return self.tree[node]
    if end < l or start > r:
        return -inf
    mid = (l + r) >> 1
    self.push_down(node)
    q_left = self.query(node << 1, start, end, l, mid)
    q_right = self.query(node << 1 | 1, start, end, mid + 1, r)
    return max(q_left, q_right)

def solve():
    n, q = map(int, input().split())
    # 初始化线段树
    sol = SegmentTree(n, [0] * (n + 1))
    output = []

    for _ in range(q):
        line = input().split()
        if line[0] == 'Add':
            l, r, v = map(int, line[1:])
            sol.update_range(1, l, r, 1, n, v)
        else:
            l, r = map(int, line[1:])
            output.append(str(sol.query(1, l, r, 1, n)))

    sys.stdout.write("\n".join(output))

if __name__ == "__main__":
    solve()

```

(至少包含有"Accepted")

#52600435提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
import sys

# 设置递归深度和快速输入
input = sys.stdin.readline
inf = float('inf')

class SegmentTree:
    def __init__(self, n, nums):
        self.n = n
        self.nums = nums
        self.tree = [0] * (4 * self.n)
        self.tag = [0] * (4 * self.n)
        # self.build(1, 1, n)

    def push_up(self, node):
        return
```

基本信息

#: 52600435  
题目: 30878  
提交人: 25n2500011619(xq)  
内存: 59752kB  
时间: 1098ms  
语言: PyPy3  
提交时间: 2026-05-21 22:23:09

## 2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

T5学习了用首末字母作为点把哈密顿路径转化成欧拉路径问题的思想，同时考虑了字典序的问题。

Q：题解的做法是将所有输入的单词按字典序从小到大排序，比如有三个单词：bab, babab, bcc，按照题解的算法会输出 bab.babab.bcc，有没有可能是 babab.bab.bcc？

在进行深度优先搜索（DFS）时，邻接表里的出边排序决定了我们先走哪条边。

- 单词 A: bab
- 单词 B: babab 根据字符串比较规则，前三个字符大家都一样是 b a b。当比较到第四个字符时，单词 A 已经结束了（相当于空字符），而单词 B 是 a。在大多数编程语言（包括 Python、C++、Java）的默认规则中，较短的前缀字符串永远小于以它为前缀的长字符串。因此：bab < babab。这也是为什么算法会优先选择走 bab。

### 最终输出路径（带点号）的字典序比较

题目要求输出的格式是以点号 . 分隔的字符串。我们直接对比这两个拼接后的最终结果：

- 路径 1（算法输出）：bab.babab.bcc
- 路径 2（hack）：babab.bab.bcc

让我们逐个字符进行对比（ASCII 码比较）：

| 索引位置 | 路径 1 的字符     | 路径 2 的字符     | 比较结果  |
|------|--------------|--------------|-------|
| 0    | b            | b            | 相同    |
| 1    | a            | a            | 相同    |
| 2    | b            | b            | 相同    |
| 3    | . (ASCII 46) | a (ASCII 97) | . < a |

会发现在第 4 个字符（索引 3）的地方分出了胜负：点号 `.` 的 ASCII 码是 46，而小写字母 `a` 的 ASCII 码是 97。

因为 `.` < `a`，所以整体字符串 `bab.babab.bcc` 的字典序是严格小于 `babab.bab.bcc` 的。

## 延伸：什么时候hack是对的？

如果假设题目要求不要点号分隔，直接拼成一个长字符串 `babbababbcc` 对比 `bababbabbcc`：

- `babbababbcc` (路径 1 直拼)
- `bababbabbcc` (路径 2 直拼) 在这个平行问题里，第 4 个字符是 `b` vs `a`，此时确实是路径 2 的字典序更小！

正是因为题目规定了用 `.` 作为分隔符，且 `.` 的 ASCII 码比所有英文字母都要小，所以“先将单词按字典序从小到大排序，DFS 贪心选择”这个算法在逻辑上是无懈可击的。它天然保证了生成出来的带点号的路径，一定具有最小字典序。

## 练习题

### 洛谷 P2515 [HAOI2010] 软件安装

SCC, Tree DP, (<https://www.luogu.com.cn/problem/P2515>)

树形依赖背包：如果选择子树里的任何节点，就必须先选择根节点  $u$ 。定义  $dp[u][j]$  表示：在以节点  $u$  为根的子树中，选择总重量恰好（或不超过） $j$  的软件，所能获得的最大价值。状态转移顺序从叶子节点开始向上。

代码：

```
import sys

sys.setrecursionlimit(2000)
def tarjan_scc(graph):
    n = len(graph)
    timer = 0
    dfn = [-1] * n
```

```

low = [-1] * n
on_stack = [False] * n
stack = []
sccs = []

def dfs(u):
    nonlocal timer
    dfn[u] = low[u] = timer
    timer += 1
    on_stack[u] = True
    stack.append(u)

    v = graph[u]
    if dfn[v] == -1:
        dfs(v)
        low[u] = min(low[u], low[v])
    elif on_stack[v]:
        low[u] = min(low[u], dfn[v])

    if dfn[u] == low[u]:
        tmp = []
        while stack:
            top = stack.pop()
            tmp.append(top)
            on_stack[top] = False
            if top == u: break
        sccs.append(tmp)

for i in range(n):
    if dfn[i] == -1: dfs(i)
return sccs

def main():
    input = sys.stdin.readline
    n, m = map(int, input().split())
    w = [0] + list(map(int, input().split()))
    v = [0] + list(map(int, input().split()))
    d = [0] + list(map(int, input().split()))

    sccs = tarjan_scc(d)
    num_scc = len(sccs)
    scc_id = [-1] * (n + 1)
    scc_w = [0] * num_scc
    scc_v = [0] * num_scc
    for i, scc in enumerate(sccs):

```

```

    for u in scc:
        scc_id[u] = i
        scc_w[i] += w[u]
        scc_v[i] += v[u]

fa = list(range(num_scc))
children = [[] for _ in range(num_scc)]
for u, v in enumerate(d):
    u_scc = scc_id[u]
    v_scc = scc_id[v]
    if u_scc != v_scc:
        fa[u_scc] = v_scc
        children[v_scc].append(u_scc)
for i in range(1, num_scc):
    if fa[i] == i:
        fa[i] = 0
        children[0].append(i)

## Tree DP
dp = [[0] * (m + 1) for _ in range(num_scc)]
# 在以节点 u 为根的子树中，选择总重量恰好（或不超过）j 的软件，所能获得的最大价值
for u in range(num_scc - 1, -1, -1):

    for j in range(scc_w[u], m + 1):
        dp[u][j] = scc_v[u]

    for v in children[u]:
        # 分组背包核心逻辑：容量外层倒序，子树分配内层顺序
        for j in range(m, scc_w[u] - 1, -1):
            # k 至少需要提供 scc_w[v] 才有意义
            for k in range(scc_w[v], j - scc_w[u] + 1):
                dp[u][j] = max(dp[u][j], dp[u][j-k] + dp[v][k])
print(max(dp[0]))

if __name__ == '__main__':
    main()

```

## 关键路径 AOE

练习了晴问中的模板题。

```

import sys
from collections import defaultdict

def topo_sort(n, edges, in_deg):
    q = [i for i in range(n) if in_deg[i] == 0]

```

```

est = [0] * n
head = 0
while head < len(q):
    u = q[head]
    head += 1
    for v, w in edges[u]:
        in_deg[v] -= 1
        if est[u] + w > est[v]:
            est[v] = est[u] + w
        if in_deg[v] == 0:
            q.append(v)
if len(q) < n:
    return None, None
return est, q

```

```

def get_critical_path(n, edges, in_deg):

```

```

    est, topo_order = topo_sort(n, edges, in_deg)
    if topo_order is None:
        return -1, None

```

```

    max_length = max(est)
    lst = [max_length] * n
    activity = defaultdict(list)
    for u in reversed(topo_order):
        for v, w in edges[u]:
            if lst[v] - w < lst[u]:
                lst[u] = lst[v] - w
            if est[u] + w == lst[v]:
                activity[u].append((v, w))

```

```

    return max_length, activity

```

```

def print_critical_path(u, activity, max_length):

```

```

    output = [u]

```

```

    def dfs(node, cnt):
        if cnt == max_length:
            print(*output, sep='->')
            return
        for v, w in sorted(activity[node]):
            output.append(v)
            dfs(v, cnt + w)
            output.pop()

```



```
dfs(u, 0)
```

```
def main():
    data = sys.stdin.read().split()
    if not data:
        return

    n = int(data[0])
    m = int(data[1])
    ptr = 2

    edges = [[] for _ in range(n)]
    in_deg = [0] * n
    for _ in range(m):
        u, v, w = map(int, data[ptr : ptr+3])
        edges[u].append((v, w))
        in_deg[v] += 1
        ptr += 3

    in_deg_copy = in_deg.copy()
    max_length, activity = get_critical_path(n, edges, in_deg_copy)

    if max_length == -1:
        print('No')
    else:
        print('Yes')
        print(max_length)
        for i in range(n):
            if in_deg[i] == 0:
                print_critical_path(i, activity, max_length)

if __name__ == "__main__":
    main()
```